



CHOKRO

Wireframe and Schema Diagrams

Android + Responsive Web
Flutter + Firebase + Node.js

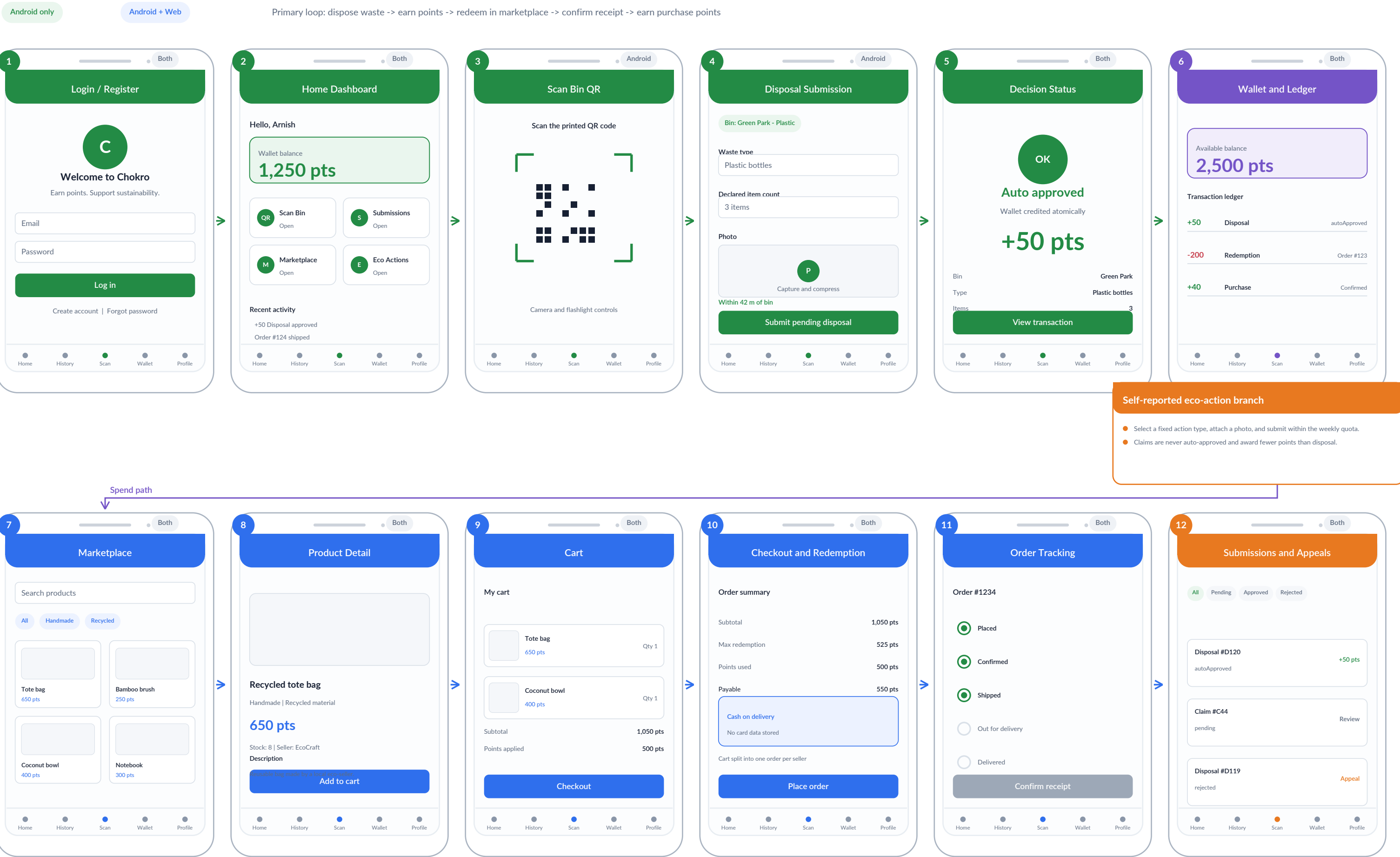
- Clear, scalable vector diagrams
- A3 landscape pages for printing
- Resident, seller and admin journeys
- Architecture and Firestore data model
- Security and trust-boundary notes

Based on Chokro Mobile Project Brief v2.0

Diagram set overview

Each diagram has been separated onto its own page to keep text and relationships readable.

01	Resident and buyer wireframe Mobile disposal, wallet, marketplace, checkout and history	Page 2
02	Seller and administrator wireframe Product, order, bin, review, policy and governance screens	Page 3
03	System architecture and trust boundary Flutter client, Firebase, trusted Node service, Gemini and FCM	Page 4
04	Schema A: Identity, verification and rewards Users, bins, disposals, claims, wallets, transactions and controls	Page 5
05	Schema B: Marketplace and governance Products, carts, orders, appeals, stats and embedded order items	Page 6
06	Implementation rules and field ownership Client-write, server-write, invariants and status machines	Page 7
Core project principles ● No client writes a wallet balance, not even an administrator. ● Every balance change is paired with a source-tagged transaction in one atomic operation. ● Uncertain screening routes to human review. It never routes directly to payout. ● Disposal is mobile-only by design; most other features are responsive on Android and web.		



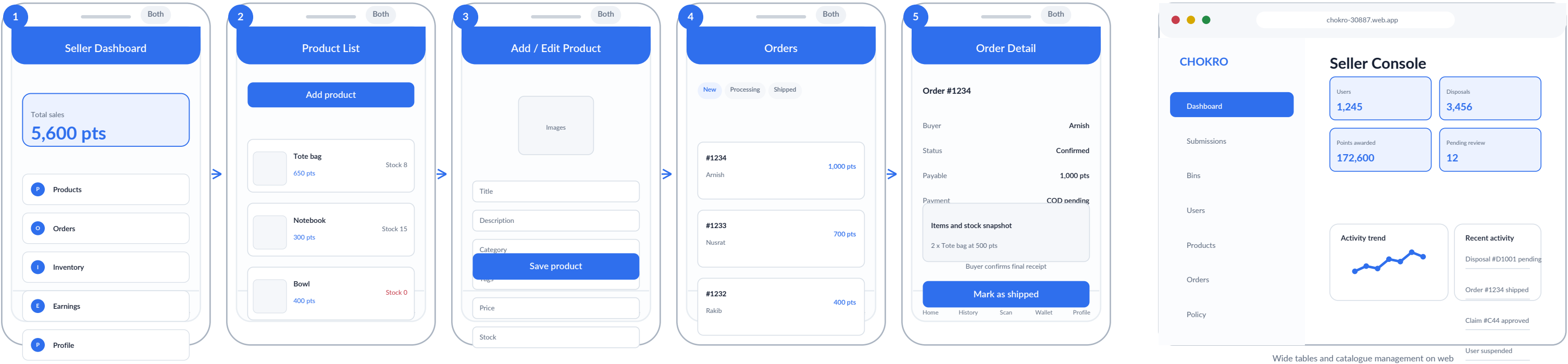
Spend path

Self-reported eco-action branch

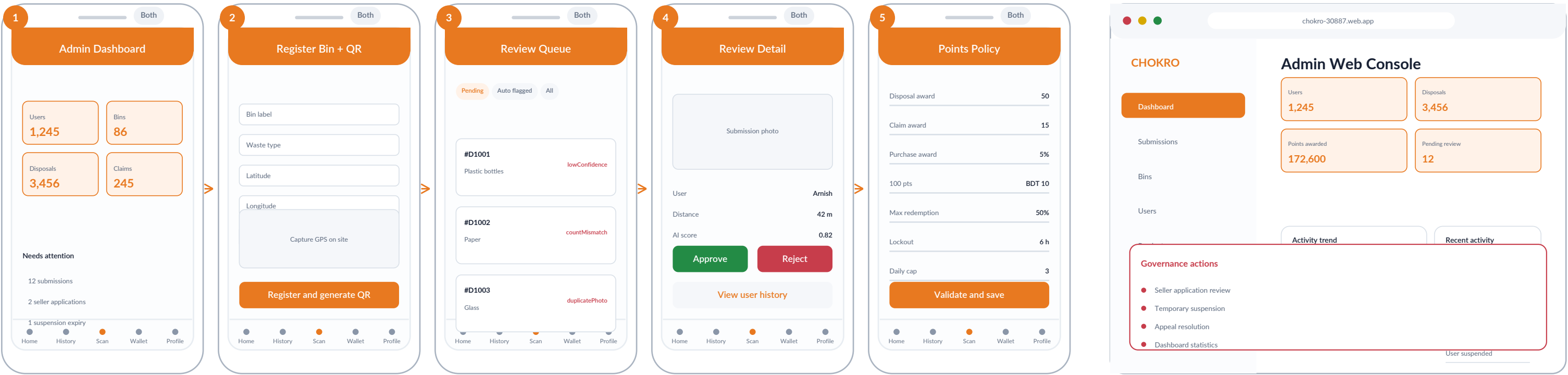
Select a fixed action type, attach a photo, and submit within the weekly quota.

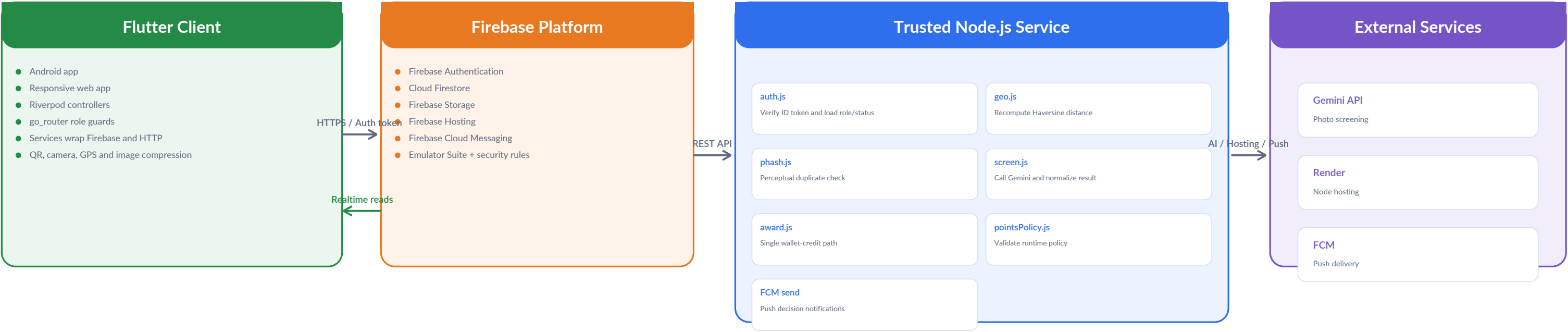
Claims are never auto-approved and award fewer points than disposal.

A. Eco-seller flow



B. Administrator flow





TRUST BOUNDARY: clients create safe pending records and read permitted data; only the trusted service decides payouts and writes server-owned fields

Write ownership and data flow

Operation	Flutter client	Firestore rules	Node service		Result
Register account	Creates user + zero wallet	Allows exact safe fields	Not required		Registration works even if Render sleeps
Submit disposal	Creates pending document	Rejects award/status fields	Verifies and decides		Auto-approve or route to review
Approve disposal	Calls review endpoint	Client update denied	Writes decision + award transaction		Wallet and ledger change atomically
Edit points policy	Calls server endpoint	Client write denied	Validates invariants and writes config		Runtime policy safely updated
Checkout	Sends item IDs and quantities	Direct privileged writes denied	Uses authoritative price and stock		Orders, stock and redemption are consistent

Security rules prove

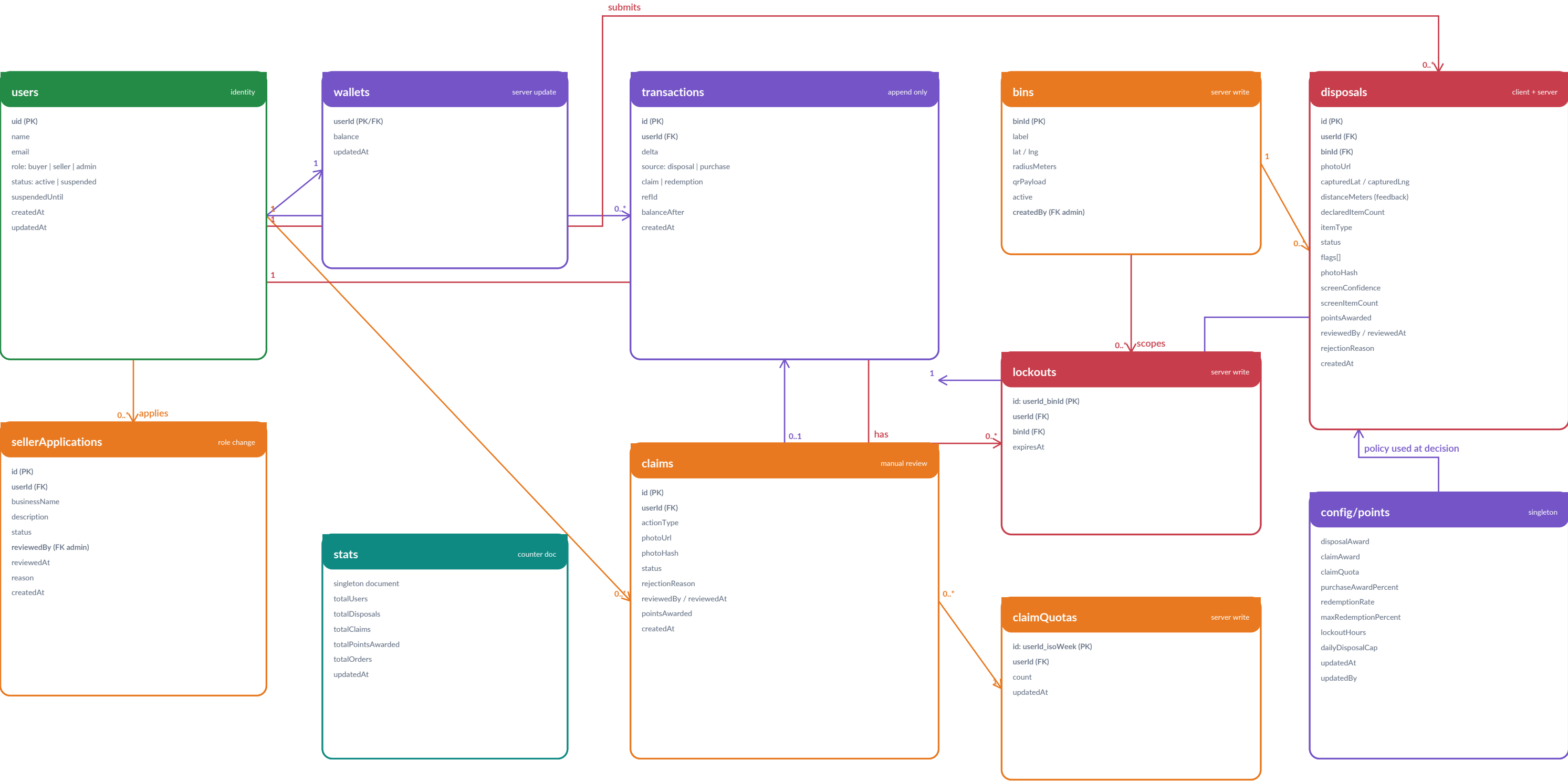
- Wallet updates and transaction writes are denied to every client role.
- Disposal creation uses an exact key set so payout fields cannot be smuggled in.
- Unknown status values fail toward pending and therefore toward no payout.

Atomic business invariants

- Every balance change and matching ledger entry are committed together.
- Checkout decrements stock in the same transaction as order creation.
- Awards and order prices are snapshotted at decision time.

Failure behavior

- Screening timeout or rate limit adds screeningUnavailable and routes to review.
- Count mismatch flags for review instead of automatic rejection.
- Rejected disposal releases its lockout so a legitimate retry is possible.



Cardinality and field ownership

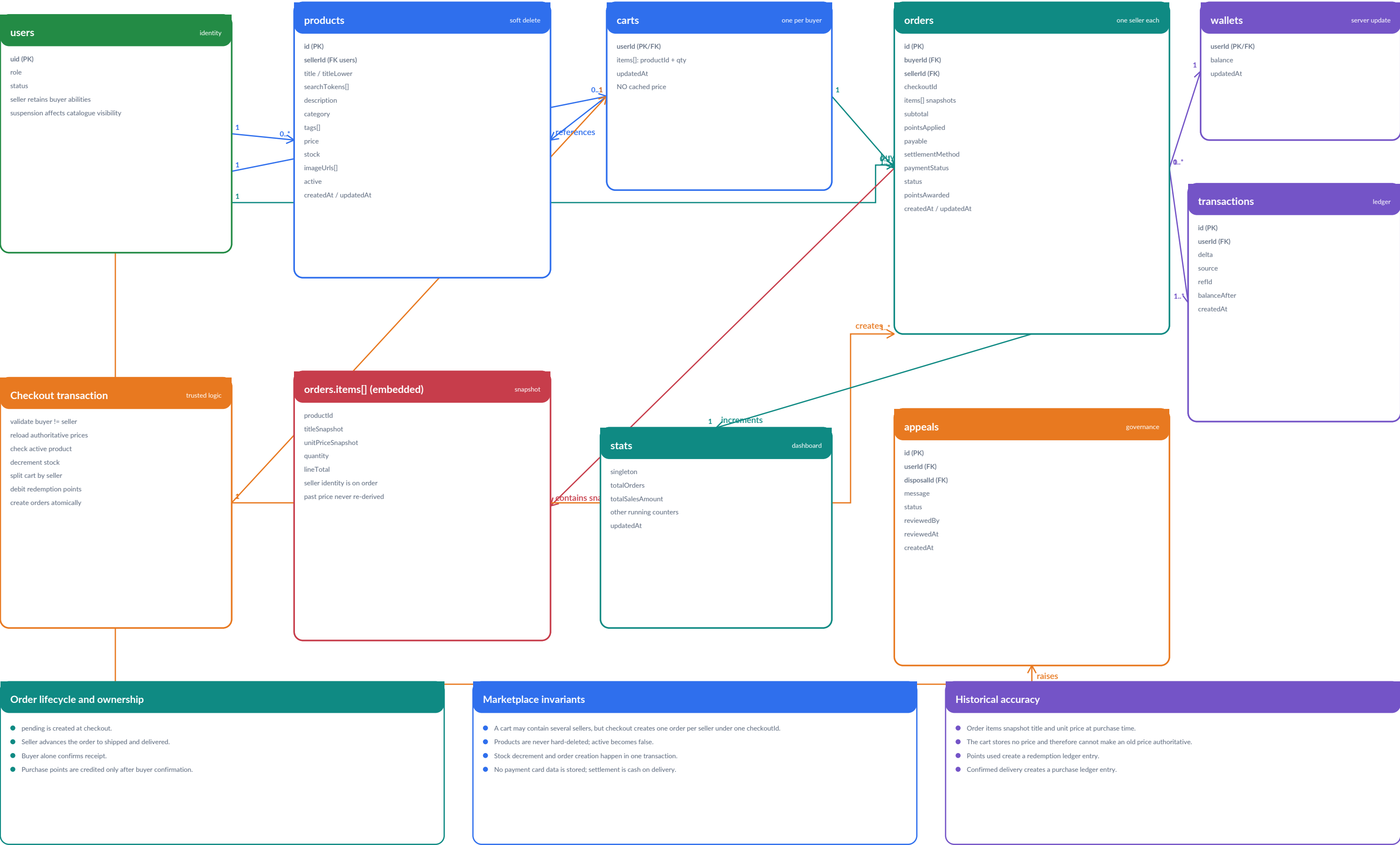
- 1 means exactly one; 0..* means zero or many.
- Blue fields originate from a safe client submission; red decision fields are server-owned.
- Firestore uses document IDs as primary keys. Foreign keys are stored as referenced IDs.
- Collections such as lockouts and claimQuotas use deterministic composite document IDs.

Status machines

- Disposal: pending -> autoApproved | manualApproved | rejected.
- Claim: pending -> approved | rejected. Claims are never auto-approved.
- Seller application: pending -> approved | rejected.
- Unknown disposal status is interpreted as pending to prevent payout.

Reward integrity

- Wallet balance is reconstructable from transactions.
- Points policy is editable, but every award is snapshotted at decision time.
- Claim quota increments on approval, not on submission.
- Stats are incremented in the same trusted operations that change the platform state.



Collection-level client access

Collection	Client read	Client create	Client update/delete	Trusted service responsibilities
users	Own; admin may read more	Own buyer profile	Name only; admin role/status	Enforce role/status for protected endpoints
wallets	Own; admin may read	Own balance = 0 at registration	Denied to everyone	Atomic balance update with transaction
transactions	Own; admin may read	Denied	Denied	Append source-tagged ledger entry
bins	Signed-in users	Denied	Denied	Register coordinates, radius and QR payload
disposals	Own; admin may read	Exact pending key set	Denied	Verify, flag, decide, credit and notify
claims	Own; admin may read	Safe pending submission	Denied	Review, quota check, credit and notify

Disposal status

- pending: wallet untouched and human review may be required.
- autoApproved: every mechanical check passed.
- manualApproved: an administrator approved after review.
- rejected: no points; reason shown; logout released.

Disposal flags

- outsideRadius
- duplicatePhoto
- countMismatch
- lowConfidence
- itemTypeMismatch
- dailyCapReached
- screeningUnavailable

Points policy defaults

- Disposal award: 50 points
- Claim award: 15 points
- Claim quota: 3 approved per ISO week
- Purchase award: 5% of payable as points
- 100 points = BDT 10
- Maximum redemption: 50% of subtotal
- Lockout: 6 hours; daily disposal cap: 3

Non-functional rules

- Primary actions within three taps.
- Paginate Firestore reads.
- Compress images and strip metadata.
- Enable offline persistence.
- No credentials in the repository.
- No credential content in server logs.
- Run dart analyze clean before device tests.

Required testing focus

- Pure Dart tests: geolocation, points policy, arithmetic and model parsing.
- Firebase Emulator tests: no client-writable balance, no client approval, exact key sets and role rules.
- Pure Node tests: verification decisions, policy validation and the single award path.

- Use a physical Android device because emulator camera and GPS can hide real permission and location problems.
- Each Jest rules suite uses a separate projectId to prevent parallel suites clearing one another.

Final continuous demonstration

- Register or sign in -> scan a printed bin QR -> capture photo and GPS -> submit pending disposal.
- Show auto-approval or manual review -> wallet increases -> disposal transaction appears.
- Browse products -> add to cart -> redeem points -> create seller-specific order.

- Seller advances status -> buyer confirms receipt -> purchase reward appears in the same wallet.
- Show four ledger source values: disposal, redemption, purchase and claim.